

FREE GUIDE

Build with AI

without building a monster

A practical guide to planning,
designing and shipping
with AI.

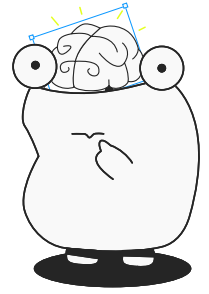


Plan

Figma

Claude Design

Claude Code



Start with *structure*

AI lets you build software faster. It does not remove the need to think. The better your planning, the better the AI output. That part has not changed.

Most weak AI builds fail because people start prompting before they understand what they are building. They ask AI to make an app. AI makes something that looks like an app. Then the problems show up — unclear journeys, messy data, drifting design, and code nobody wants to touch.

Don't start with AI. Start with structure. Then let AI do the boring parts.

THE WORKFLOW

01

Plan & brief

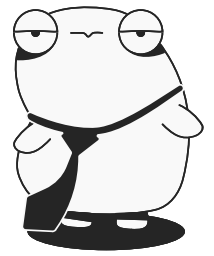
02

Figma & Claude Design

03

Code, ship, test

Plan → Figma → Claude Design → Claude Code → GitHub → Deploy. This guide walks through each step, in the order we use it ourselves at SmpIco.



Plan before *you prompt*

Before you open Claude, Cursor, Lovable or any other build tool, answer the boring questions. The boring ones are where the value lives.

Be specific about who it is for. Operations managers in construction companies tracking jobs, documents and people across multiple sites — not 'small businesses'. Specific prompts produce specific software. Vague prompts produce vague software.

AI loves adding things. You need to decide what this version is supposed to prove, and stop building when it does.

FOUR QUESTIONS BEFORE THE FIRST PROMPT

01

Who is it for?

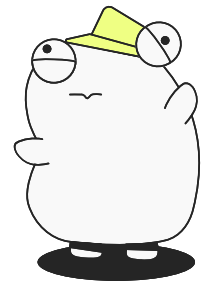
02

What problem are you solving?

03

The first useful journey?

And then — what does 'done' actually mean? Users understand the idea. The main journey works. The AI output is useful. A customer would want to try it again. That is enough for a first version.



Write a *build brief*

A brief gives the AI something useful to work from. It does not need to be long. It needs to be clear. One page. Ten lines.

Product. User. Problem. Main journey. Screens. Data. Roles. Design direction. Out of scope. Success criteria. That is it. Keep it on screen while you prompt and re-read it whenever the AI starts inventing features nobody asked for.

One page can save you a week. Probably one of those weeks where everyone says the prototype is 'nearly there' while quietly losing the will to live.

THE BRIEF, IN THREE PARTS

01

Product, user, problem

02

Journey & screens

03

Scope & success

The brief is the spine of the whole build. Write it before the first prompt. Update it when something real changes. If the team disagrees about the brief, you have not yet started the right project.

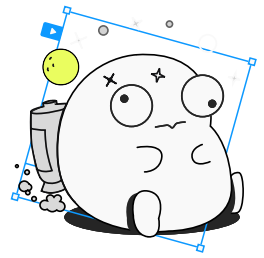
Smpl^{co}

PART 01

Plan, design, *decide.*

The boring questions are
where the value lives.
The fun parts come after.





Design the *journey*

Figma is where you make the idea visible before you ask AI to build it. Not perfect — visible. Real content where you can. AI tools read intent from the design, so vague screens produce vague output. Pretty empty templates produce confident wireframes for a product nobody will use.

Design at minimum the start, the main action, the result, and three states people forget about until production: empty, error, success. Then a basic design system underneath — buttons, inputs, cards, tables, navigation, colours, type, spacing. Enough that the product does not reinvent itself on every screen.

You do not need a massive component library. You need enough consistency that nobody can tell which screens were generated on Tuesday versus Thursday.

THE THREE STATES PEOPLE SKIP

Empty

What people see before there is data.

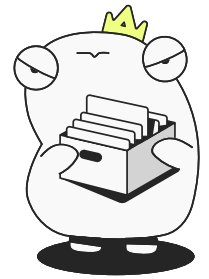
Loading

Spinners, skeletons, progress signals.

Error

When things break, gracefully.

Skip those three and your AI build will look great in the demo and break the moment a real user does anything other than the happy path. Which they will, the moment you stop watching.



Operationalise *the system*

This is the step that stops your AI-built product looking like every other AI-built product. You know the look. Default purple gradients. Glassmorphism nobody asked for. Dashboards that feel like a 2021 crypto startup. Fine for a throwaway. Less fine when you are showing investors, customers, or partners.

Claude Design turns your Figma system into actual rules — components, layout, typography, colours, spacing, cards, tables, empty and error states, interactions. Not visual inspiration. Implementation guidance Claude Code can follow without inventing things you did not ask for.

Make the AI build inside your design system, not beside it.

WHAT THE AI WILL DO BY DEFAULT

Purple

Default gradients on every screen.

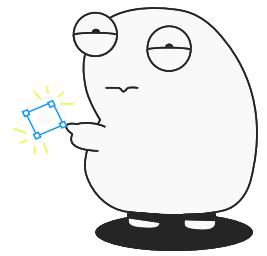
Shadows

Dropped randomly on every card.

Colours

Hex codes you never specified.

Your product should feel like one coherent experience, not a pile of screens generated by different prompts on different days. Claude Design is how you make that explicit instead of hopeful.



Build with *Claude Code*

Once the plan, the design and the design rules are clear, Claude Code can turn them into working software. App structure, components, page wiring, refactors, bug fixes, reusable UI, missing-state checks, README, GitHub prep, API connections. The boring parts write themselves now.

Do not ask 'can you build the whole app?' Ask: build the main journey from this brief, this Figma design, and these design rules. Start with structure, routes, reusable components, mock data, main journey only. No extra features, no new visual styles. And before editing files, show me the plan, the components you'll use, anything new, and your assumptions.

You want AI to behave like a good junior developer, not a caffeinated raccoon with access to your repo.

PRINCIPLES THAT KEEP IT BORING

01

Plan before code

02

Main journey first

03

No new visual styles

Plan first. Then build. The output is dramatically better when AI has to ask permission before showing off, and you spend less time deleting things that should never have shipped.

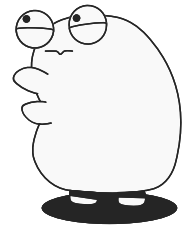
Smpl[•]

PART 02

Ship, secure, *sustain.*

Working code is the start.
Working product is the work.





Ship the *thing*

If the code matters, it lives in GitHub. Stable main branch, a new branch for each change, review before merging, deploy from main. Add a README — what the product does, how to run it, what env vars it needs, what is not done yet. Future-you will thank present-you.

Then deploy somewhere accessible. Vercel is common; Netlify, Render, Railway, Fly, Cloudflare Pages all work. The European pack: Scaleway, OVHcloud, Hetzner, Clever Cloud. The flow is the same — connect repo, env vars, deploy, test the URL, attach a domain, test again.

For early apps, run on a subdomain like `app.yourcompany.com`. Cleaner, safer, less stressful than touching your main site.

THE BORING THREE STEPS

01

GitHub

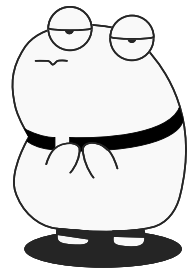
02

Deploy

03

Domain

That is the boring infrastructure that turns a prototype into a product. Skip it and you have a demo. Demos do not survive their first contact with real users.



Data & *security*

AI-generated apps end up with hardcoded data, fake permissions and secrets in the wrong place more often than you would think. Even for a prototype, check. Five minutes now beats a frantic message thread later.

Keys in env, not source. .env in gitignore. .env.example committed. User roles explicit. Sensitive data not stored where it should not be. Hosting region you can defend. Logs you understand. Failure modes you have imagined before they happen.

If you cannot tell a customer where their data is processed, where it is stored, who can access it, and whether it is used for training — you are not yet ready to onboard them.

EUROPEAN ALTERNATIVES, IF YOU NEED THEM

Hosting

Scaleway · OVHcloud ·
Hetzner

AI

Mistral · Aleph Alpha

Code

GitLab · Codeberg

You do not always need European alternatives. But for public sector, regulated industries, or anything sensitive, knowing what exists is the difference between a deal and a polite no.



Build what *matters*

One simple rule. Build what makes you different. Integrate what does not. You probably do not need to write your own login, payments, email, analytics, CMS or file storage — unless those are your actual product.

Focus on the workflow, the experience, the decision logic, the data model, the proprietary layer. The bit users would actually pay for. And before adding any AI feature, ask the question that decides everything: is AI the product, or just a feature? They are very different commitments.

AI is brilliant at moving fast. It is not brilliant at knowing what matters. That part is still on you.

BEFORE YOU SHIP, HAVE THESE

01

Real users tried it

02

GitHub & README done

03

Secrets and roles handled

Plan the product. Design the journey. Operationalise the design system. Use AI to build the first useful version. Put the code somewhere safe. Ship it. Test with real people. That is how you build with AI without building a monster.



Want help with *your AI build?*

We've built 125+ digital products with founders, scaleups and global brands. If you want a sanity check on what you're building, or someone to build it with you, we're here.

[Book a free 30-min call](#)

SAY HELLO

Andreas & Mike

Co-founders, SmplCo

andreas@smpl.as

[smpl.as](#) · [LinkedIn](#) · [Blog](#)

